ExpresionInvalidaException
+ n(String mensaje): ExpresionInvalidaExceptio
~ r(mensaje): supe
+ n(String mensaje, Throwable causa): ExpresionInvalidaExceptio
~ r(mensaje, causa): supe
<<enum>>
TipoIntervalo
<<enum>>
TipoPuntoCritico
Punto
+ o(double x, double y): Punt
<<enum>>
TipoFuncion
Dominio
+ o(Double desde, Double hasta): Domini
Evaluador
~ tokens: String[]
~ placeholders: String[]
+ r(String expresion): Evaluado
+ evaluar(double x): double
~ r(): siguienteCaracte
- normalizar(String expr): String
~ e("raiz(", "sqrt("): .replac
~ r(int i = 0; i < tokens.length; i++): fo
~ r(int i = 0; i < tokens.length; i++): fo
- siguienteCaracter(): void
- consumir(int charToEat): boolean
~ r(): siguienteCaracte
~ f(caracter == charToEat): i
~ r(): siguienteCaracte
- analizarExpresion(double x): double
~ r(...): fo
- analizarTermino(double x): double
~ r(...): fo
- analizarUnary(double x): double
- analizarPotencia(double x): double
- analizarFactor(double x): double
~ r(): siguienteCaracte
~ r(): siguienteCaracte
~ r(): siguienteCaracte
Intervalo
- tipoIntervalo: TipoIntervalo
+ o(Double inicio, Double fin, TipoIntervalo...): Interval
PuntoCritico
- tipoPuntoCritico: TipoPuntoCritico
+ o(double x, double y, TipoPuntoCritico tip...): PuntoCritic
~ r(x, y): supe
DerivadorSimbolico
~ analizador: Analizador
~ ast: Nodo
~ derivado: Nodo
~ simplificado: Nodo
~ analizador: Analizador
~ ast: Nodo
~ primeraDerivada: Nodo
~ segundaDerivada: Nodo
~ analizador: Analizador
~ ast: Nodo
~ sl: Nodo
~ sr: Nodo
~ tiL: TermInfo
~ tiR: TermInfo
~ sl: Nodo
~ sr: Nodo
~ tiL: TermInfo
~ tiR: TermInfo
~ sl: Nodo
~ sr: Nodo
~ sl: Nodo
~ sr: Nodo
~ base: Nodo
~ sb: Nodo
~ se: Nodo
~ sb: Nodo
~ se: Nodo
~ argumento: Nodo
~ derivadoInterno: Nodo
~ derivadoExterno: Nodo
~ sa: Nodo
~ base: Nodo
~ m: NodoMultiplicacion
~ nodo: Nodo
~ nodo: Nodo
~ nodo: Nodo
~ c: char
~ n: Nodo
~ arg: Nodo
+ derivar(String expresion): String
+ derivarSegunda(String expresion): String
+ toLaTeX(String expresion): String
- normalizar(String expresion): String
~ e("raiz(", "sqrt("): .replac
~ derivar(): Nodo
~ simplificar(): Nodo
~ toLaTeX(): String
~ toMathExpression(): String
~ e(double v): NodoConstant
+ derivar(): Nodo
+ simplificar(): Nodo
+ toMathExpression(): String
+ toLaTeX(): String
+ derivar(): Nodo
+ simplificar(): Nodo
+ toMathExpression(): String
+ toLaTeX(): String
~ a(Nodo l, Nodo r): NodoSum
+ derivar(): Nodo
+ simplificar(): Nodo
~ f(sl instanceof NodoConstante && sr instan...): i
+ toMathExpression(): String
+ toLaTeX(): String
~ a(Nodo l, Nodo r): NodoRest
+ derivar(): Nodo
+ simplificar(): Nodo
~ f(sl instanceof NodoConstante && sr instan...): i
+ toMathExpression(): String
+ toLaTeX(): String
~ n(Nodo l, Nodo r): NodoMultiplicacio
+ derivar(): Nodo
+ simplificar(): Nodo
~ f(sl instanceof NodoConstante && sr instan...): i
~ f(sl instanceof NodoConstante): i
~ f(v == -1): i
~ f(sr instanceof NodoConstante): i
+ toMathExpression(): String
+ toLaTeX(): String
~ n(Nodo l, Nodo r): NodoDivisio
+ derivar(): Nodo
+ simplificar(): Nodo
~ f(sr instanceof NodoConstante): i
~ f(sl instanceof NodoDivision): i
~ f(sl instanceof NodoConstante && sr instan...): i
+ toMathExpression(): String
+ toLaTeX(): String
~ a(Nodo b, Nodo e): NodoPotenci
+ derivar(): Nodo
~ f(se instanceof NodoConstante): i
~ f(sb instanceof NodoConstante): i
+ simplificar(): Nodo
~ f(se instanceof NodoConstante): i
~ f(sb instanceof NodoPotencia): i
~ f(se instanceof NodoConstante): i
+ toMathExpression(): String
+ toLaTeX(): String
~ f(base instanceof NodoVariable
base ins...): i
~ n(String n, Nodo a): NodoFuncio
+ derivar(): Nodo
~ h(nombre): switc
+ simplificar(): Nodo
+ toMathExpression(): String
+ toLaTeX(): String
~ o(double c, Nodo b): TermInf
~ extract(Nodo n): TermInfo
~ f(n instanceof NodoMultiplicacion): i
~ f(m.izquierda instanceof NodoConstante): i
~ n(String s): Analizado
~ analizar(): Nodo
~ analizarSumaResta(): Nodo
~ analizarMultDiv(): Nodo
~ analizarUnary(): Nodo
~ analizarPotencia(): Nodo
~ analizarFactor(): Nodo
~ f(c == '('): i
~ f(c == 'x'): i
ResultadoAnalisis
- puntosCriticos: PuntoCritico[]
- puntosInflexion: PuntoCritico[]
- intervalosCrecimiento: Intervalo[]
- intervalosDecrecimiento: Intervalo[]
- intervalosConcavidad: Intervalo[]
+ s(): ResultadoAnalisi
+ s(PuntoCritico[] puntosCriticos, PuntoCrit...): ResultadoAnalisi
+ s(String resumen): ResultadoAnalis
+ intervalosConcavidad(): Intervalo[]
- formatearIntervalo(Intervalo intervalo): String
+ tieneResultados(): boolean
+ agregarMensajeAccesibilidad(String mensaje): void
~ f(this.mensajesAccesibilidad == null): i
~ d(mensaje): this.mensajesAccesibilidad.ad
GestorHistorial
- instancia: GestorHistorial
- historial: List<String>
- ii(): GestorHistoria
~ f(instancia == null): i
+ agregarExpresion(String expresion): void
~ d(expresion): historial.ad
+ eliminarExpresion(String expresion): void
~ e(expresion): historial.remov
+ limpiarHistorial(): void
~ r(): historial.clea
ValidadorExpresion
~ puntosPrueba: double[]
~ caracter: char
~ caracter: char
~ caracter: char
~ actual: char
~ siguiente: char
~ ultimo: char
+ validar(String expresion): void
~ s(expresion): validarCaractere
~ s(expresion): validarParentesi
~ s(expresion): validarSintaxisBasic
~ f(expresion): validarIndeterminacionGloba
- validarIndeterminacionGlobal(String expresion): void
~ r(double x : puntosPrueba): fo
~ f(!algunDefinido): i
~ f(algunInfinito && !algunNaN): i
- validarCaracteres(String expresion): void
~ r(int i = 0; i < expr.length(): fo
- validarParentesis(String expresion): void
~ r(int i = 0; i < expresion.length(): fo
~ f(caracter == '('): i
~ f(balanceParentesis < 0): i
~ f(balanceParentesis > 0): i
- validarSintaxisBasica(String expresion): void
- esOperadorBinario(char c): boolean
CalculoNumericoException
+ n(String mensaje): CalculoNumericoExceptio
~ r(mensaje): supe
+ n(String mensaje, Throwable causa): CalculoNumericoExceptio
~ r(mensaje, causa): supe
TestLog
~ evalLn: Evaluador
~ evalLog: Evaluador
+ main(String[] args): void
~ e(): e.printStackTrac
Funcion
- tipoFuncion: TipoFuncion
- dominioFuncion: Dominio
+ n(int idFuncion, String expresion, String ...): Funcio
AnalizadorFuncion
~ evaluador: Evaluador
~ puntosCriticos: PuntoCritico[]
~ puntosInflexion: PuntoCritico[]
~ intervalosCrecimiento: Intervalo[]
~ intervalosDecrecimiento: Intervalo[]
~ intervalosConcavidad: Intervalo[]
~ raicesPrimeraDerivada: List<Double>
~ raicesSegundaDerivada: List<Double>
~ intervalosMonotonia: Intervalo[]
~ crecientes: List<Intervalo>
~ decrecientes: List<Intervalo>
~ resultado: ResultadoAnalisis
~ singularidades: List<Singularidad>
~ raices: List<Double>
~ puntos: PuntoCritico[]
~ tipo: TipoPuntoCritico
~ puntos: PuntoCritico[]
~ divisiones: List<Double>
~ intervalos: List<Intervalo>
~ tipo: TipoIntervalo
~ divisiones: List<Double>
~ intervalos: List<Intervalo>
~ tipo: TipoIntervalo
~ funciones: String[]
~ funciones: String[]
~ asintotas: List<Double>
~ indefinidos: List<Double>
~ sb: StringBuilder
~ sb: StringBuilder
~ singularidades: List<Singularidad>
~ unicas: List<Singularidad>
~ calcular(double x): double
+ analizar(String expresion, boolean calcPuntosCrit...): ResultadoAnalisis
+ analizarEnRango(String expresion, double minX, double ma...): ResultadoAnalisis
~ r(expresion): ValidadorExpresion.valida
~ f(calcPuntosCriticos
calcMaxMin): i
~ f(calcMaxMin): i
~ f(calcInflexion): i
~ f(calcIntervalos): i
~ r(Intervalo intervalo : intervalosMonotoni...): fo
~ d(intervalo): crecientes.ad
~ d(intervalo): decrecientes.ad
~ f(calcConcavidad): i
~ r(double x = minX; x <= maxX; x += step): fo
~ s(singularidades, resultado): procesarSingularidade
~ f(siempreDerivadaCero): i
~ s(new PuntoCritico(2.0)): resultado.setPuntosCritico
~ o(new Intervalo(0)): resultado.setIntervalosCrecimient
~ o(new Intervalo(0)): resultado.setIntervalosDecrecimient
~ n(new PuntoCritico(0)): resultado.setPuntosInflexio
~ d(new Intervalo(0)): resultado.setIntervalosConcavida
~ d("Esta es una función constante. No tiene...): resultado.agregarMensajeAccesibilida
~ n(new PuntoCritico(0)): resultado.setPuntosInflexio
~ d(new Intervalo(0)): resultado.setIntervalosConcavida
~ s(new PuntoCritico(0)): resultado.setPuntosCritico
~ d("Esta es una función lineal. No tiene pu...): resultado.agregarMensajeAccesibilida
~ d("Esta es una función trigonométrica.”): resultado.agregarMensajeAccesibilida
~ d("Debido a su periodicidad, los puntos cr...): resultado.agregarMensajeAccesibilida
~ d("El dominio está restringido y presenta ...): resultado.agregarMensajeAccesibilida
~ d("Las funciones cotangente y cosecante ti...): resultado.agregarMensajeAccesibilida
~ d("El dominio está restringido en múltiplo...): resultado.agregarMensajeAccesibilida
~ d("Esta es una función logarítmica.”): resultado.agregarMensajeAccesibilida
~ d("El dominio está restringido: el argumen...): resultado.agregarMensajeAccesibilida
~ d("Presenta una asíntota vertical en el va...): resultado.agregarMensajeAccesibilida
~ d("Al contener una raíz par, el dominio es...): resultado.agregarMensajeAccesibilida
~ d("Esta es una función racional. Puede pre...): resultado.agregarMensajeAccesibilida
~ n(new PuntoCritico(0)): resultado.setPuntosInflexio
~ d(msg): resultado.agregarMensajeAccesibilida
~ f(grado > 0): i
~ f(grado % 2 == 0): i
~ d("Al ser de grado par, posee al menos un ...): resultado.agregarMensajeAccesibilida
~ d("Al ser de grado impar, posee al menos u...): resultado.agregarMensajeAccesibilida
~ encontrarRaicesFuncionDerivada funcion, double minX, do...): List<Double>
~ r(double x = minX + step; x <= maxX; x += ...): fo
~ d(raiz): raices.ad
~ biseccion(FuncionDerivada funcion, double a, doubl...): double
~ r(int i = 0; i < MAX_ITERACIONES_BISECCION...): fo
~ f(fa * fc < 0): i
~ clasificarPuntosCriticos(Evaluador evaluador, List<Double> raices): PuntoCritico[]
~ r(int i = 0; i < raices.size(): fo
~ f(segundaDerivada > 0): i
~ crearPuntosCriticos(Evaluador evaluador, List<Double> raices...): PuntoCritico[]
~ r(int i = 0; i < raices.size(): fo
~ calcularIntervalosMonotonia(Evaluador evaluador, PuntoCritico[] punt...): Intervalo[]
~ d(minX): divisiones.ad
~ r(PuntoCritico pc : puntosCriticos): fo
~ d(maxX): divisiones.ad
~ calcularIntervalosConcavidad(Evaluador evaluador, PuntoCritico[] punt...): Intervalo[]
~ d(minX): divisiones.ad
~ r(PuntoCritico pi : puntosInflexion): fo
~ d(maxX): divisiones.ad
~ derivada(Evaluador f, double x): double
~ segundaDerivada(Evaluador f, double x): double
~ esPolinomio(String expr): boolean
~ r(String func : funciones): fo
~ esRacional(String expr): boolean
~ esIrracional(String expr): boolean
~ esTrigonometrica(String expr): boolean
~ r(String func : funciones): fo
~ esLogaritmica(String expr): boolean
~ Singularidad(double x, TipoSingularidad tipo): record
~ procesarSingularidades(List<Singularidad> singularidades, Resul...): void
~ r(int i = 0; i < asintotas.size(): fo
~ d(", "): sb.appen
~ d("}"): sb.appen
~ d(" "): sb.appen
~ d(", ..."): sb.appen
~ d("). Esto puede ocurrir en raíces de n/ime...): sb.appen
~ identificarSingularidades(Evaluador f, double minX, double maxX): List<Singularidad>
~ r(double x = minX; x <= maxX; x += step): fo
~ r(Singularidad s : singularidades): fo
~ r(Singularidad u : unicas): fo
~ d(s): unicas.ad
~ detectarGradoProbable(Evaluador f, double minX, double maxX): int
~ esDerivadaConstante(Evaluador f, int orden, double minX, dou...): boolean
~ r(double x = minX + 2.* step; x < maxX; x ...): fo
~ calcularDerivadaOrdenN(Evaluador f, int orden, double x): double
~ h(orden): switc
<<interface>>
IFuncion
~ analizar(Funcion funcion, boolean calcPuntosCriti...): ResultadoAnalisis
VentanaUtil
~ webView: WebView
~ engine: WebEngine
+ render(String latex): WebView
~ t(90): webView.setPrefHeigh
~ t(85): webView.setMinHeigh
~ t(200): webView.setMaxHeigh
~ e("-fx-background-color: transparent;"): webView.setStyl
~ t(html): engine.loadConten
FormulaRenderer
~ webView: WebView
~ engine: WebEngine
+ render(String latex): WebView
~ t(90): webView.setPrefHeigh
~ t(85): webView.setMinHeigh
~ t(200): webView.setMaxHeigh
~ e("-fx-background-color: transparent;"): webView.setStyl
~ t(html): engine.loadConten
GraficadorCanvas
- canvas: Canvas
- gc: GraphicsContext
- evaluador: Evaluador
- resultado: ResultadoAnalisis
~ steps: double[]
+ s(double width, double height): GraficadorCanva
~ s(): configurarEvento
- configurarEventos(): void
~ t(this::manejarDesplazamiento): canvas.setOnScrol
~ d(this::manejarMousePresionado): canvas.setOnMousePresse
~ d(this::manejarMouseArrastrado): canvas.setOnMouseDragge
~ d(this::manejarMouseLiberado): canvas.setOnMouseRelease
~ d(this::manejarMouseMovido): canvas.setOnMouseMove
+ graficar(String expresion, ResultadoAnalisis resu...): void
~ n("GraficadorCanvas.graficar llamado con: ...): System.out.printl
~ r(): dibuja
- dibujar(): void
~ l(Color.WHITE): gc.setFil
~ a(): dibujarCuadricul
~ s(): dibujarEje
~ f(evaluador != null): i
~ n(): dibujarFuncio
~ f(resultado != null): i
~ s(): dibujarPuntosCritico
~ o(): dibujarTitul
- dibujarCuadricula(): void
~ e(Color.LIGHTGRAY): gc.setStrok
~ h(0.5): gc.setLineWidt
~ e(screenX, 0, screenX, height): gc.strokeLin
~ e(0, screenY, width, screenY): gc.strokeLin
- calcularStep(double rango): double
~ r(double step : steps): fo
~ f(step >= targetStep): i
- dibujarEjes(): void
~ e(Color.BLACK): gc.setStrok
~ h(2): gc.setLineWidt
~ f(minX <= 0 && maxX >= 0): i
~ e(screenX, 0, screenX, height): gc.strokeLin
~ l(Color.BLACK): gc.setFil
~ n("textAlignment.CENTER): gc.setTextAlig
~ l("Y", screenX + 15, 15): gc.fillTex
~ f(minY <= 0 && maxY >= 0): i
~ e(0, screenY, width, screenY): gc.strokeLin
~ l("X", width - 15, screenY - 5): gc.fillTex
~ a(): dibujarMarcasEje
- dibujarMarcasEjes(): void
~ l(Color.BLACK): gc.setFil
~ n("textAlignment.CENTER): gc.setTextAlig
~ f(minY <= 0 && maxY >= 0): i
~ f(minX <= 0 && maxX >= 0): i
- dibujarFuncion(): void
~ h(2.5): gc.setLineWidt
~ r(double x = minX; x <= maxX; x += step): fo
~ f(previscreenX != null): i
~ e(prevScreenX, prevScreenY, screenX, scree...): gc.strokeLin
~ n("Error INESPERADO graficando en x=" + x ...): System.out.printl
~ e(): e.printStackTrac
- dibujarPuntosCriticos(): void
- dibujarPunto(double x, double y, Color color, String ...): void
~ l(color): gc.setFil
~ l(screenX - 5, screenY - 5, 10, 10): gc.fillOva
~ e(Color.WHITE): gc.setStrok
~ h(2): gc.setLineWidt
~ l(screenX - 5, screenY - 5, 10, 10): gc.strokeOva
~ l(color): gc.setFil
~ t(etiqueta, screenX + 8, screenY - 8): gc.fillTex
- dibujarTitulo(): void
~ f(expresion != null): i
~ l(Color.BLACK): gc.setFil
~ n("textAlignment.LEFT): gc.setTextAlig
- xAPantalla(double x): double
- yAPantalla(double y): double
- pantallaAX(double screenX): double
- pantallaAY(double screenY): double
- manejarDesplazamiento(ScrollEvent event): void
~ r(): dibuja
~ e(): event.consum
- manejarMousePresionado(MouseEvent event): void
~ r(javafx.scene.Cursor.CLOSED_HAND): canvas.setCurso
- manejarMouseArrastrado(MouseEvent event): void
~ f(isDragging): i
~ r(): dibuja
- manejarMouseLiberado(MouseEvent event): void
~ r(javafx.scene.Cursor.DEFAULT): canvas.setCurso
- manejarMouseMovido(MouseEvent event): void
+ resetearZoom(): void
~ r(): dibuja
+ establecerTamanio(double width, double height): void
~ f(width <= 0
height <= 0): i
~ f(width > 4000
height > 4000): i
~ h(width): canvas.setWidt
~ h(height): canvas.setHeigh
~ f(evaluador != null): i
~ r(): dibuja
HistorialFuncionesControlador
- vboxHistorial: VBox
- btnLimpiar: Button
- calcularControlador: CalcularControlador
~ history: List<String>
~ itemContainer: HBox
~ btnFunc: Button
~ stage: Stage
~ btnDelete: Button
~ ii(): cargarHistoria
- onLimpiarHistorial(ActionEvent event): void
- cargarHistoria(): void
~ r(String expression : history): fo
~ t(Pos.CENTER_LEFT): itemContainer.setAlignmen
~ e("-fx-padding: 5; -fx-border-color: #eee;...): itemContainer.setStyl
~ h(Double.MAX_VALUE): btnFunc.setMaxWidt
~ e("-fx-background-color: transparent; -fx-...): btnFunc.setStyl
~ w(btnFunc, Priority.ALWAYS): HBox.setHgro
~ r(event -> { if (calcularControlador != nu...): btnFunc.setOnActio
~ r(expression): calcularControlador.cargarYCalcula
~ e(): stage.clos
~ e("-fx-background-color: #ff5252; -fx-text...): btnDelete.setStyl
~ ii(): cargarHistoria
TecladoVirtual
- popup: Popup
- inputField: TextField
~ root: VBox
~ grid: GridPane
~ btnAC: Button
~ btnDel: Button
~ btnOk: Button
~ btn: Button
~ btn: Button
~ point: Point2D
+ ii(): TecladoVirtua
~ e(true): this.popup.setAutoHid
~ ii(): inicializarUI
- inicializarUI(): void
~ g(10): root.setSpacin
~ p(5): grid.setHga
~ p(5): grid.setVga
~ t(Pos.CENTER): grid.setAlignmen
~ n(grid, "sin", "sin(", 0, 0): agregarBoto
~ n(grid, "cos", "cos(", 1, 0): agregarBoto
~ n(grid, "tan", "tan(", 2, 0): agregarBoto
~ n(grid, "log", "log(", 3, 0): agregarBoto
~ n(grid, "ln", "ln(", 4, 0): agregarBoto
~ n(grid, "√", "sqrt(", 5, 0): agregarBoto
~ n(grid, "(", "(", 0, 1): agregarBoto
~ n(grid, "7", "7", 2, 1): agregarBoto
~ n(grid, "8", "8", 3, 1): agregarBoto
~ n(grid, "9", "9", 4, 1): agregarBoto
~ n(grid, "/", "/", 5, 1): agregarBoto
~ n(grid, "x", "x", 0, 2): agregarBoto
~ n(grid, "^", "^", 1, 2): agregarBoto
~ n(grid, "4", "4", 2, 2): agregarBoto
~ n(grid, "5", "5", 3, 2): agregarBoto
~ n(grid, "6", "6", 4, 2): agregarBoto
~ n(grid, "*", "*", 5, 2): agregarBoto
~ n(grid, "π", "pi", 0, 3): agregarBoto
~ n(grid, "e", "e", 1, 3): agregarBoto
~ n(grid, "1", "1", 2, 3): agregarBoto
~ n(grid, "2", "2", 3, 3): agregarBoto
~ n(grid, "3", "3", 4, 3): agregarBoto
~ n(grid, "-", "-", 5, 3): agregarBoto
~ r(): inputField.clea
~ d(btnAC, 0, 4): grid.ad
~ d(btnDel, 1, 4): grid.ad
~ n(grid, ")", ")", 2, 4): agregarBoto
~ n(grid, ".", ".", 3, 4): agregarBoto
~ d(btnOk, 4, 4, 2, 1): grid.ad
~ h(Double.MAX_VALUE): btnOk.setMaxWidt
- agregarBoto(GridPane grid, String label, String valu...): void
~ d(btn, col, row): grid.ad
- crearStyledButton(String text, String styleClass): Button
- insertarTexto(String texto): void
~ t(texto): inputField.appendTex
~ t(newText): inputField.setTex
~ s(): inputField.requestFocu
- borrarCaracter(): void
~ f(caretPos > 0): i
~ t(newText): inputField.setTex
~ t(caretPos - 1): inputField.positionCare
~ s(): inputField.requestFocu
+ mostrar(TextField inputField, Node anchorNode): void
~ e(): popup.hid
~ w(anchorNode, x, y): popup.sho
VistaPrincipalControlador
- instancia: VistaPrincipalControlador
- hBoxLogo: HBox
- txtBuscar: TextField
- btnSalir: Button
- btnInicio: ToggleButton
- btnCalcular: ToggleButton
- btnAyuda: ToggleButton
- panelCarga: AnchorPane
- circleLogo: Circle
- img: Image
- menuToggleGroup: ToggleGroup
~ loader: FXMLLoader
~ vista: AnchorPane
~ a(RUTA_INICIO): cargarVist
~ e(): iniciarBotonesToggl
~ r(): aplicarMascaraCircula
- aplicarMascaraCircular(): void
- iniciarBotonesToggle(): void
- p(menuToggleGroup): btnInicio.setToggleGrou
- p(menuToggleGroup): btnCalcular.setToggleGrou
- p(menuToggleGroup): btnAyuda.setToggleGrou
~ d(true): btnInicio.setSelecte
~ f(newValue != null): i
~ a(RUTA_INICIO): cargarVist
~ a(RUTA_CALCULAR): cargarVist
~ a(RUTA_AYUDA): cargarVist
- cargarVista(String rutaVista): void
~ r(vista, 0.0): AnchorPane.setTopAncho
~ r(vista, 0.0): AnchorPane.setBottomAncho
~ r(vista, 0.0): AnchorPane.setLeftAncho
~ r(vista, 0.0): AnchorPane.setRightAncho
- onBtnInicio(ActionEvent actionEvent): void
~ d(true): btnInicio.setSelecte
- onBtnCalcular(ActionEvent actionEvent): void
~ d(true): btnCalcular.setSelecte
- onBtnAyuda(ActionEvent actionEvent): void
~ d(true): btnAyuda.setSelecte
- onBtnSalir(ActionEvent actionEvent): void
~ t(0): System.exi
- onBuscar(ActionEvent event): void
~ d(true): btnAyuda.setSelecte
~ d(true): btnAyuda.setSelecte
~ d(true): btnCalcular.setSelecte
~ d(true): btnInicio.setSelecte
~ d(true): btnCalcular.setSelecte
- contienePalabras(String query, String... palabras): boolean
~ r(String palabra : palabras): fo
- normalizar(String texto): String
~ m(): tri
+ navegarACalcular(): void
~ d(true): btnCalcular.setSelecte
App
~ loader: FXMLLoader
~ scene: Scene
+ start(Stage stage): void
~ e(scene): stage.setScen
~ e("IntRexPoint"): stage.setTitl
~ d(true): stage.setMaximize
~ w(): stage.sho
+ main(String[] args): void
~ h(): launc
AyudaControlador
FuncionImpl
- analizador: AnalizadorFuncion
+ l(): FuncionImp
+ analizar(Funcion funcion, boolean calcPuntosCriti...): ResultadoAnalisis
~ f(funcion == null): i
~ f(inicio >= fin): i
ReproduceBug
~ analizador: AnalizadorFuncion
~ expre: String[]
~ res: ResultadoAnalisis
+ main(String[] args): void
~ r(String expr : expre): fo
~ n("Testing: " + expr): System.out.printl
~ n(" Expr: " + expr): System.out.printl
~ n(" MSG: " + msg): System.out.printl
~ n("----------"): System.out.printl
CalcularControlador
+ vboxResultado: VBox
- chkPuntosCriticos: RadioButton
- chkIntervalos: RadioButton
- chkMaxMin: RadioButton
- chkPuntoInflexion: RadioButton
- chkConcavidad: RadioButton
- txtFuncion: TextField
- txtMinX: TextField
- txtMaxX: TextField
- btnTeclado: Button
- btnCalcular: Button
- btnBorrar: Button
- vboxResultadosTexto: VBox
- contenedorGrafica: HBox
- analizador: AnalizadorFuncion
- graficadorCanvas: GraficadorCanvas
- tecladoVirtual: TecladoVirtual
~ resultado: ResultadoAnalisis
~ alertBox: VBox
~ lbl: Label
~ section: VBox
~ formulasContainer: VBox
~ scrollDerivadas: ScrollPane
~ section: VBox
~ section: VBox
~ section: VBox
~ section: VBox
~ emptyBox: VBox
~ lblEmpty: Label
~ card: VBox
~ lblTitle: Label
~ hbox: HBox
~ lblKey: Label
~ lblValue: Label
~ label: Label
~ alert: Alert
~ loader: FXMLLoader
~ root: Parent
~ controller: HistorialFuncionesControlador
~ stage: Stage
- calcular(ActionEvent event): void
~ s(): limpiarResultado
~ f(minX >= maxX): i
~ a("Rango Inválido", "El valor mínimo debe ...): mostrarAlert
~ a("Rango Inválido", "Ingrese valores numér...): mostrarAlert
~ s(resultado): mostrarResultadosTextuale
~ n("Intentando graficar: " + expresion): System.out.printl
~ r(expresion, resultado): graficadorCanvas.grafica
~ n("Gráfica completada exitosamente"): System.out.printl
~ e(): e.printStackTrac
- validarEntrada(String expresion): boolean
~ a("Campo Vacío", "Debe ingresar una funció...): mostrarAlert
~ a("Sin Opciones", "Debe seleccionar al men...): mostrarAlert
- mostrarResultadosTextuales(ResultadoAnalisis resultado): void
~ e("-fx-background-color: #ffff3cd; -fx-bord...): alertBox.setStyl
~ e("-fx-text-fill: #856404; -fx-font-weight:...): lbl.setStyl
~ t(true): lbl.setWrapTex
~ y(ScrollPane.ScrollBarPolicy.AS_NEEDED): scrollDerivadas.setHbarPolic
~ y(ScrollPane.ScrollBarPolicy.NEVER): scrollDerivadas.setVbarPolic
~ e(true): scrollDerivadas.setPannabl
~ t(false): scrollDerivadas.setFitToHeigh
~ t(180): scrollDerivadas.setMinHeigh
~ e("-fx-background-color: transparent; -fx-...): scrollDerivadas.setStyl
~ f(hasCrecimiento
hasDecr): i
~ f(hasCrecimiento): i
~ f(hasDecr): i
~ t(javafx.geometry.Pos.CENTER): emptyBox.setAlignmen
~ t(true): lblEmpty.setWrapTex
~ e("-fx-text-fill: #999; -fx-font-style: it...): lblEmpty.setStyl
- crearSeccion(String title): VBox
~ h(Double.MAX_VALUE): lblTitle.setMaxWidt
- crearEtiquetaDual(String key, String value): HBox
~ t(javafx.geometry.Pos.CENTER_LEFT): hbox.setAlignmen
- crearEtiquetaFormula(String formula): Label
~ h(Double.MAX_VALUE): label.setMaxWidt
- formatearIntervalo(com.espoch.inflexpoint.modelos.entidades...): String
- limpiarResultados(): void
- borrar(ActionEvent event): void
~ r(): txtFuncion.clea
~ d(false): chkPuntosCriticos.setSelecte
~ d(false): chkIntervalos.setSelecte
~ d(false): chkMaxMin.setSelecte
~ d(false): chkPuntoInflexion.setSelecte
~ d(false): chkConcavidad.setSelecte
~ t("-10"): txtMinX.setTex
~ t("10"): txtMaxX.setTex
~ s(): limpiarResultado
- mostrarTeclado(ActionEvent event): void
~ f(tecladoVirtual == null): i
~ r(txtFuncion, btnTeclado): tecladoVirtual.mostra
- mostrarAlerta(String titulo, String mensaje): void
~ e(titulo): alert.setTitl
~ t(null): alert.setHeaderTex
~ t(mensaje): alert.setContentTex
~ o(alert): VentanaUtil.aplicarIcon
~ ti(): alert.showAndWai
+ cargarYCalcular(String expresion): void
~ t(expresion): txtFuncion.setTex
~ d(true): chkPuntosCriticos.setSelecte
~ d(true): chkIntervalos.setSelecte
~ d(true): chkMaxMin.setSelecte
~ d(true): chkPuntoInflexion.setSelecte
~ d(true): chkConcavidad.setSelecte
~ r(null): calcula
- verHistorial(ActionEvent event): void
~ r(this): controller.setCalcularControlado
~ e("Historial de Funciones"): stage.setTitl
~ y(Modality.APPLICATION_MODAL): stage.initModalit
~ o(stage): VentanaUtil.aplicarIcon
~ w(): stage.sho
~ e(): e.printStackTrac
InicioControlador
~ principal: VistaPrincipalControlador
- onComenzarAnalisis(): void
~ f(principal != null): i
~ r(): principal.navegarACalcula